

Reinforcement Learning Assignment 2025 - 2026

Coding classical and modern RL algorithms

Isidoro Tamassia, Giuseppe Marra

April 2026

1 Introduction

In this assignment, you will embark on a coding journey to learn how to implement the fundamental RL methods studied throughout the course, as well as some of the modern Deep-RL methods. You will test these algorithms on a set of grid-world environments, which provide a good starting point for understanding the impact of different algorithm design choices on the agent's behavior.

You will be working with **Python**. The assignment is mainly based on three frameworks:

- **OpenAI Gym**, a standard API for Reinforcement Learning (RL) environments, in the `gymnasium` package (<https://gymnasium.farama.org/api/env/>)¹.
- **Minihack**, a sandbox framework for easily designing rich and diverse environments for RL, based on the game of NetHack and providing an OpenAI Gym interface (<https://minihack.readthedocs.io/en/latest/index.html>).
- **PyTorch**, one of the de-facto standard libraries for building deep learning systems (<https://docs.pytorch.org/docs/stable/index.html>). This will be needed for the deep-RL exercises.

We provide the necessary material in the repository root. **All** the tasks need to be implemented by filling in the indicated functions in the provided template. In other words, look for each `#TODO` comment present in the codebase, and implement it. These can be either complete algorithms or environment implementations, or only some parts of them, depending on the exercise. Moreover, you will need to perform some hyperparameter tuning to train both the tabular and deep-RL methods. You can find all the details in the remainder of this document.

2 Rules

The assignment is individual. You must implement the required algorithms by yourself, and you are not allowed to share code with your peers. Your assignment must adhere to general scientific standards of credit attribution. More details on this can be found at <https://www.kuleuven.be/english/education/plagiarism/>. Moreover, **you are not allowed to use generative AI tools**, such as ChatGPT, Claude Code, Copilot, etc, to complete this assignment. **Clear evidence of the use of such tools in your solutions may make you ineligible to take the exam.**

The only parts of the repository that you are supposed to modify are:

¹Gymnasium is the maintained fork of the [original Gym API](#), which is unmaintained since 2021.

- The code components where a **#TODO** is present. Please do not remove the comment delimiters `### YOUR SOLUTION STARTS HERE` and `### END OF YOUR SOLUTION`, as they are used to extract your solution and include them in the report.
- The hyperparameters for training tabular and deep agents, as more thoroughly explained in [section 7](#).

Anything else should stay unchanged, meaning that IF you modify it, THEN your solutions may be marked as incorrect even if the tests pass. Of course, the test scripts themselves are among the things you must not modify.

Read and understand the given parts of code that are not part of your solution as they give the context in which you must implement your solution. The task specification in the section below is often only introductory to the task and the code is the main specification.

2.1 Questions

If you have any questions about the assignment:

- You can ask **technical** questions to your peers, but **sharing part of the solution is prohibited**;
- Ask the teaching assistant and the professor during **teaching moments**;
- Only if the previous two are not possible, you can reach out by email.

3 Submission

You will need to submit two files in Toledo:

- A zip of the folder of your assignment (this is the folder where you run your algorithms and tests), including the training weights. The principle is that we must be able to rerun your tests and obtain the same results obtained by you.
- The report generated by:

```
docker compose run --rm py python tools/report.py --student-id 12345678
```

where `student-id` is your student number. The report is generated in the folder:

```
report/report_<your-student-number>.pdf
```

Important

In essence, this means that IF your implementation passes the provided tests, AND you did not modify components which were not supposed to be modified, THEN **you are automatically admitted to the written exam once you submit a report with 100% pass rate**. The report is a receipt that you fulfill the aforementioned criteria.

Notice that we reserve the right to call you in for a **10-minute oral discussion** in case something is suspicious with your submission.

4 Deadline

Submit required materials on Toledo

Before June 1st, 23:59

5 Installation

This project is based on Docker. If you are not familiar with Docker, you can find more information and installation instructions on the Docker website <https://www.docker.com/>. After installing Docker, go to the root of the repo and run:

```
docker compose build
```

That's it. Note that the current Dockerfile installs a CPU-only version of PyTorch, but that is sufficient to complete the assignment.

To run any script you simply run:

```
docker compose run --rm py python <relative-path-to-script>
```

The repo is structured as follows:

- `commons.py` is a collection of utility functions. Nothing for you to modify here.
- `minihack_env.py` contains the collection of MiniHack environments used throughout the assignment. Nothing for you to modify here.
- `quickstart.py` contains small code snippets to get started with the `gymnasium` and `MiniHack` APIs. Nothing for you to modify here.
- The folder `task1` contains the files for Task 1, described in [subsection 6.1](#).
- The folder `task2` contains the files for Task 2, described in [subsection 6.2](#).
- The folder `task3` contains the files for Task 3, described in [subsection 6.3](#).
- The folder `train` contains three training scripts:
 - `train_tabular.py` trains the tabular agents (MC, SARSA, Q-learning) and saves the corresponding Q-tables, which are used by the performance tests (see [section 7](#)). It also contains the hyperparameter configurations you are expected to tune.
 - `train_dqn.py` trains the DQN agent and saves the network weights, which are used by the performance tests. It also contains the hyperparameter configurations you are expected to tune.
 - `train_ppo.py` does the same as the above but for PPO.
- The folder `trained_weights` will be created automatically the first time you run a training script. It contains the Q-tables and network weights saved by the training scripts, which are loaded by the performance tests.
- The folder `tests` contains the test scripts used to check your solutions. A full description of how to run them and what they check is provided in [section 7](#).
- The folder `tools` contains `report.py`, the script used to generate your submission report (see [section 3](#)).

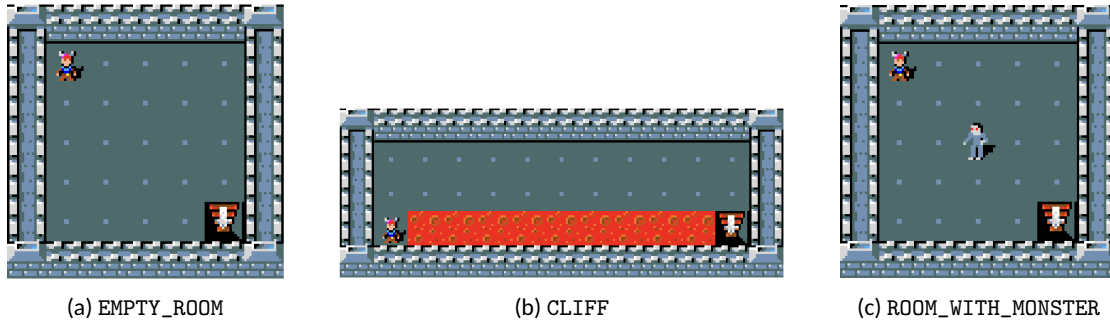


Figure 1: The three MiniHack environments included in the codebase. In all the environments, the action space consists on the four cardinal actions ($\uparrow$, $\downarrow$, $\leftarrow$, $\rightarrow$). In all the environments, the agent only gets a reward +1 once reaching the goal, and always zero otherwise. In all the environments, moving towards the wall results in staying in the same state. In CLIFF, moving towards the lava kills the agent and terminates the episode. This is called CLIFF although there's no cliff: that's because it's inspired by https://gymnasium.farama.org/environments/toy_text/cliff_walking/.

6 Tasks

6.1 Task 1: Environments and Dummy Agents

In this first task, you will start by exploring the Gymnasium API, the MiniHack framework, and how simple agents interact with these environments. All the subtasks of this task can be found in `task1/dummy.py`.

1. Encode a variable-size ($n \times m$) `GridWorldEnv` in Gymnasium, with 4 available actions (the cardinal movements), where the agent starts in the $(0, 0)$ coordinate (top-left corner of the grid), the goal is always in the $(n-1, m-1)$ coordinate (bottom-right corner), and the agent receives a fixed reward of -1 for each transition. The action space and observation space are already specified in the code. To do this, complete the following methods of the class `GridWorldEnv` in `task1/dummy.py`:
 - `reset`, which sets (and returns) the initial state of the agent (i.e., its coordinate position on the grid).
 - `step`, which determines the next state and reward given the previous state and an action, and returns the next state, reward, and termination/truncation flags.
 - `render`, which returns a string representation of the current grid.
2. Encode a stochastic variable-size ($n \times m$) `SlipperyGridWorldEnv` in Gymnasium. This environment has the same grid layout, start state, goal state, action encoding, and reward function as `GridWorldEnv`, but transitions are noisy. If the agent chooses an action, then with probability p_{success} the intended action is executed, with probability $(1 - p_{\text{success}})/2$ the agent turns left relative to the intended direction, and with probability $(1 - p_{\text{success}})/2$ the agent turns right relative to the intended direction. Use the random number generator provided by Gymnasium through `self.np_random`. The action space and observation space are already specified in the code. To do this, complete the following methods of the class `SlipperyGridWorldEnv` in `task1/dummy.py`:
 - `reset`, which sets (and returns) the initial state of the agent. If `options` contains `"start_pos"`, then the agent must start from that coordinate; otherwise it starts from $(0, 0)$.

- `step`, which samples the executed action according to the stochastic transition model, computes the next state and reward, and returns the next state, reward, and termination/truncation flags.
 - `render`, which returns a string representation of the current grid.
3. Implement a `GridWorldHardCodedAgent` which solves `GridWorldEnv` optimally by leveraging knowledge of the goal position. Rewards do not matter in this exercise. To do this, complete the following method in the `GridWorldHardCodedAgent` class in `task1/dummy.py`:
 - `act`, which takes the current state as input and returns the next optimal move based on the goal position.
 4. Implement a `MiniHackEmptyHardCodedAgent` which solves the environment in [Figure 1a](#). Rewards do not matter in this exercise. You may hardcode a policy that follows any optimal safe path, for example first moving straight down and then right, or first right and then down. To do this, complete the following method in the `MiniHackEmptyHardCodedAgent` class in `task1/dummy.py`:
 - `act`, which takes the current state as input and returns the next move.
 5. Implement a `MiniHackCliffHardCodedAgent` which solves the environment in [Figure 1b](#). Rewards do not matter in this exercise. To do this, complete the following method in the `MiniHackCliffHardCodedAgent` class in `task1/dummy.py`:
 - `act`, which takes the current state as input and returns the next move.
 6. Implement the generic interaction loop in `AbstractRLTask`. This method should run several episodes of interaction between an agent and an environment and collect the per-episode returns. To do this, complete the following method in the `AbstractRLTask` class in `task1/dummy.py`:
 - `interact`, which executes `n_episodes` episodes, optionally truncates each episode after `max_steps_per_episode` steps, calls `agent.onEpisodeEnd(...)` at the end of each episode, and returns the list of per-episode (undiscounted) returns.

6.2 Task 2: Classical Reinforcement Learning

In this task, you will implement three classical RL agents and complete the shared tabular-agent utilities. All the tasks you need to complete can be found in the `task2` folder:

1. Shared tabular-agent components, in `task2/tabular_agent.py`:
 - Implement the ϵ -greedy policy in `choose_next_action` of `TabularMinihackAgent`: with probability ϵ select a uniformly random action (exploration), otherwise select the action with the highest Q-value (exploitation).
 - Implement the linear ϵ -decay schedule in `update_epsilon` of `TabularMinihackAgent`. After each episode, ϵ is reduced linearly from ϵ_{start} to ϵ_{end} over a fixed number of episodes:

$$\epsilon_t = \epsilon_{\text{start}} - t \frac{\epsilon_{\text{start}} - \epsilon_{\text{end}}}{\text{epsilon_decay_episodes}}.$$

This schedule is shared by all three tabular agents.

2. Monte Carlo On-policy control (MC-control), in `task2/mc-agents.py`:

- Implement `act` in `MCAgent`. Unlike TD agents, Monte Carlo does *not* learn during the episode — it must accumulate the sequence of transitions into an episode buffer and wait until the episode ends before updating.
 - Implement `onEpisodeEnd` in `MCAgent`. Because Monte Carlo requires the full return from the episode before it can update, this is where learning is triggered. After updating, the episode buffer must be cleared.
 - Implement `learn` in `MCAgent`, which applies the Monte Carlo update rule: iterate backward through the episode buffer to compute the discounted return G_t at each visited state-action pair, and update $Q(s, a)$ using an incremental average weighted by visit counts.
3. Temporal Difference On-policy control (SARSA), in `task2/td-agents.py`:
- Implement `act` and `onEpisodeEnd` in the `TDAgent` base class. In contrast to Monte Carlo, TD methods perform a *bootstrapped* update immediately after every transition, without waiting for the episode to end. These two lifecycle methods control when `learn(...)` is called; they are shared by both SARSA and Q-learning, so you only need to implement them once.
 - Implement the SARSA update rule in `learn` of `SARSAOnPolicyAgent`: given a transition (s, a, r, s', a') , update $Q(s, a)$ toward the target $r + \gamma Q(s', a')$, where a' is the action *actually selected* in s' by the current policy (on-policy).
4. Temporal Difference Off-policy control (Q-learning), in `task2/td-agents.py`:
- Implement the Q-learning update rule in `learn` of `QLearning`: given a transition (s, a, r, s') , update $Q(s, a)$ toward the target $r + \gamma \max_{a'} Q(s', a')$, where the maximum is over all actions regardless of the behaviour policy (off-policy).
 - Implement `choose_next_action` in `QLearning`. This differs from the base-class version: when the agent is in learning mode it uses ϵ -greedy exploration; when it is *not* in learning mode (i.e. during evaluation), exploration must be fully disabled so the agent always acts greedily.

You will need to train MC, SARSA, and Q-learning on the three environments in [Figure 1](#), and the testers will use the resulting Q-tables as explained in [section 7](#).

6.3 Task 3: Deep Reinforcement Learning

In this task, you will complete the implementations of two popular Deep-RL algorithms. All the tasks you need to complete can be found in the `task3` folder:

1. Deep Q-Networks (DQN):
 - Implement the **soft target network update** in `update_target_network` of `DeepMinihackAgent` (`task3/deep_agent.py`). After each update step, move each target parameter slightly toward the corresponding online-network parameter:

$$\theta_{\text{target}} \leftarrow \tau \theta + (1 - \tau) \theta_{\text{target}}$$

where $\tau \in [0, 1]$. When $\tau = 1$, this becomes a full copy (hard update).

- Implement the **DQN loss** in `learn` of `DQNAgent` (`task3/dqn.py`): compute the MSE between the current Q-values and the TD target. Use the online network to compute $Q(s, a)$ for the actions taken, and use the frozen target network to compute

$$r + \gamma \max_{a'} Q_{\text{target}}(s', a')$$

without backpropagating through the target term.

2. Proximal Policy Optimization (PPO), in `task3/ppo.py`:

- Implement the **Generalized Advantage Estimation (GAE)** recursion in `update_policy` of `PPOTask`. GAE estimates how much better an action was than the policy's average at a given state, balancing bias and variance through λ . Iterate backward through the rollout: at each step t , compute the TD residual

$$\delta_t = r_t + \gamma V(s_{t+1})(1 - d_{t+1}) - V(s_t),$$

then accumulate the advantage recursively as

$$A_t = \delta_t + \gamma \lambda (1 - d_{t+1}) A_{t+1},$$

and compute the returns as $A_t + V(s_t)$. Note that `update_policy` contains other computations; locate the `#TODO` corresponding to this exercise. You can find more about GAE in the original paper: <https://arxiv.org/abs/1506.02438>.

- Implement the **PPO clipped surrogate objective** in `update_policy` of `PPOTask`. Compute the probability ratio

$$r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)},$$

then define the policy loss using the clipped objective

$$\min(r_t(\theta) A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) A_t).$$

Note that when this objective is turned into a loss (minimised), the `min` becomes a `max`. As above, locate the corresponding `#TODO` in `update_policy`.

You will need to train both DQN and PPO on the three environments in [Figure 1](#), and the testers will use the resulting weights as explained in [section 7](#).

7 Testing your solutions

The testing scripts are provided in the `tests` folder. From the repository root, you can run a full test file with²:

```
docker compose run --rm py pytest -vv tests/test1.py
docker compose run --rm py pytest -vv tests/test2.py
docker compose run --rm py pytest -vv tests/test3.py
```

To run all tests at once, use:

```
docker compose run --rm py pytest -vv tests/
```

To run a single specific test `<name_of_test>` in `test<N>.py`, use:

```
docker compose run --rm py pytest -vv tests/test<N>.py::name_of_test
```

²The flag `-vv` simply makes the tester output more verbose, e.g. tells you more precisely which tests failed.

7.1 Testing Dummy Agents (task 1)

The results of the tests in `tests/test1.py` only depend on your completion of the missing code chunks in `task1/dummy.py`. Once you have completed them, run:

```
docker compose run --rm py pytest -vv tests/test1.py
```

If all tests pass, you are good to go. Otherwise, you can inspect which tests failed to understand which code components you need to modify.

7.2 Testing Tabular Agents Performance (task 2)

There are two kinds of tests on tabular agents: **correctness tests**, which check that core components of your implementation are (likely) correct, and **performance tests**, which check that the overall algorithm-hyperparameters combinations solve the given minihack tasks (up to a certain return threshold). The point is that if your implementations are correct AND you choose sensible hyperparameters, ALL tests should pass.

Correctness tests: These tests only check for the semantic/syntactic correctness of your implemented code chunks, e.g. by checking that a certain function returns the expected output given a dummy input. They do not depend on any hyperparameter (i.e., you don't have to perform any tuning to pass these tests, only implement the required code from the tasks).

You can run all the correctness tests for task 2 with the following command:

```
docker compose run --rm py pytest -vv tests/test2.py -m correctness
```

Performance tests: Tabular agents' performance is tested by running evaluation episodes with pre-trained Q-tables. The training script uses the `AbstractRLTask` interaction loop you implemented in Task 1. To train an agent after implementing the missing parts of the code, do the following:

- Choose your hyperparameters in `train/train_tabular.py`; comments clearly indicate which arguments you are supposed to tune and which ones you must not modify.
- Run the training script, passing the algorithm and environment as arguments. For example:

```
docker compose run --rm py python train/train_tabular.py \
    --algorithm=q_learning --env=empty-room
```

Once training is finished, the Q-table is saved to `trained_weights/tabular/<agent>/<env_name>.pkl`.

- Repeat for each combination of algorithm in `{q_learning, sarsa, monte_carlo}` and environment in `{empty-room, cliff-minihack, room-with-monster}` (9 training runs in total).

The testers will load the Q-tables from the corresponding folders and run evaluation episodes on the corresponding environment. This means the outcome of the tests will depend on both the missing parts of the algorithms' implementations that you have to complete, AND on the chosen hyperparameters.

You can run all the performance tests for task 2 with the following command:

```
docker compose run --rm py pytest -vv tests/test2.py -m performance
```


If all tests pass, you are good to go. Otherwise, you can inspect which tests failed to understand which code components you need to modify, OR whether you need to tune hyperparameters better. In general, IF all correctness tests pass, THEN it is likely that the problem is the current setting of hyperparameters, and you should try changing them and retraining the model.

7.3 Testing Deep Agents Performance (task 3)

Like for tabular agents, there are two kinds of tests on deep agents: **correctness tests**, which check that core components of your implementation are (likely) correct, and **performance tests**, which check that the overall algorithm-hyperparameters combinations solve the given minihack tasks (up to a certain return threshold). The point is that if your implementations are correct AND you choose sensible hyperparameters, ALL tests should pass.

Correctness tests: These tests only check for the semantic/syntactic correctness of your implemented code chunks, e.g. by checking that a certain function returns the expected output given a dummy input. They do not depend on any hyperparameter (i.e., you don't have to perform any tuning to pass these tests, only implement the required code from the tasks).

You can run all the correctness tests for task 3 with the following command:

```
docker compose run --rm py pytest -vv tests/test3.py -m correctness
```

Performance tests: Deep RL agents' performance is tested by running several evaluation episodes with pre-trained weights. To train an agent <agent> (either dqn or ppo) after implementing the missing parts of the code, do the following:

- Go to `train/train_<agent>.py` and choose your hyperparameters; comments clearly indicate which ones you are supposed to tune (modify) and which ones you should/must not touch. From the script, you can also select the training environment.
- Run:

```
docker compose run --rm py python train/train_<agent>.py
```

Once the training is finished, this will save the weights in `trained_weights/<agent>/<env_name>.pt`

- Repeat this training for each algorithm in {dqn, ppo} and environment in {EMPTY_ROOM, CLIFF, ROOM_WITH_MONSTER} (so you'll have to train 6 times in total).

The testers will load the weights from the corresponding folders and run some evaluation episodes on the corresponding environment.

You can run all the performance tests for task 3 with the following command:

```
docker compose run --rm py pytest -vv tests/test3.py -m performance
```

If all tests pass, you are good to go. Otherwise, you can inspect which tests failed to understand which code components you need to modify, OR whether you need to tune hyperparameters better. In general, IF all correctness tests pass but the training of your algorithm does not pick up (and therefore, tests with the corresponding weights fail), THEN it is likely that the problem is the current setting of hyperparameters, and you should try changing them and retraining the model.